

Conception d'un système intelligent pour le diagnostic automatique d'un nouveau diabète

Elhadji Abdoulaye Diankha, étudiant en maitrise technologie de l'information à l'uqo

Résumé

Les systèmes de santé intelligents sont devenus indispensables dans le domaine des sciences de la vie et jouent un rôle de plus en plus important dans la transformation des informations existantes en connaissances exploitables. La mise au point d'un système de santé intelligent pour le diagnostic et la prédiction automatisés du diabète, utilisant l'apprentissage automatique, vise à faciliter le travail des médecins spécialisés en diabétologie en leur permettant d'obtenir rapidement et efficacement les résultats des patients. Auparavant, une fois le diagnostic posé manuellement, les tâches que le médecin devait effectuer pour établir un résultat étaient fastidieuses. C'est pourquoi la détection et le diagnostic automatiques du diabète à l'aide de systèmes de santé intelligents gagnent en importance. Dans le cadre de nos travaux, nous avons conçu un modèle intelligent pour le diagnostic automatique du diabète. Nous avons entraîné sur notre ensemble de données plusieurs algorithmes d'apprentissage supervisé tels que la régression logistique, la forêt aléatoire, le classificateur multicouche ou le perceptron, et la machine à vecteurs de support, et avons constaté que la forêt aléatoire est la plus optimisée et la plus précise à des fins de classification. Elle a produit la plus grande exactitude de 98,07%, une précision, un rappel, un score F1 de 98% et une perte logarithmique de 0,03 en utilisant la méthode de sélection des caractéristiques mRMR et une division de test de 0,2. Il a produit un rappel de 97%, une précision de 97 %, un score F1 de 97%, une exactitude de 96,79% et une perte logarithmique de 0,11 sur une division de test de 0,3 avec la sélection de caractéristiques mRMR. En outre, le modèle proposé a obtenu de meilleurs résultats que la plupart des modèles.

Mots-clés; Machine Learning; Disease Diagnosis; Random Forest Algorithm; Electronic Health Records.

I. INTRODUCTION

La bioscience est un domaine qui évolue de concert depuis des décennies . Ce domaine innovant numérique joue un rôle essentiel dans l'évolution de la médecine, en élevant considérablement son niveau de sophistication. Presque tout est numérisé dans les centres hospitaliers des opérations chirurgicales guidée par des technologies qui sauvent des vies aux diagnostics et détections précoces moins fastidieux et plus précis, la numérisation des outils médicaux est là pour durer. Raison pour laquelle de nos jours les chercheurs spécialisés dans ce domaine essaient toujours de se montrer proactif ne cessant de nous impressionner à travers leurs découvertes qui continue à améliorer considérablement les les soins de santé surtout l'analyse des rapports de diagnostic et l'organisation de l'historique des traitements des patients.

Le diabète, tel qu'il est défini par l'organisation mondiale de la santé (OMS), est une maladie chronique résultant d'une production inadéquate d'insuline par le par le pancréas ou d'une utilisation inefficace de l'insuline par l'organisme . Il s'agit d'une maladie incurable, classée en deux grandes catégories : le diabète de type 1 et le diabète de type 2. Le diabète de type 1 se caractérise par une production insuffisante d'insuline , nécessitant l'administration quotidienne d'insuline exogène. À l'inverse, le diabète de type 2 résulte d'une diminution de la réactivité de l'organisme à l'insuline . Les manifestations cliniques comprennent la cétonurie, la fatigue, la faim excessive, l'augmentation de la soif, la miction fréquente, l'inexpérience et l'absence d'un

traitement adéquat, la perte de poids inexpiquée, troubles visuels, irritabilité, retard de cicatrisation des plaies et autres symptômes. Avec le temps, le diabète peut avoir des effets néfastes sur le système cardiovasculaire, le système vasculaire, les structures oculaires, la fonction rénale et les nerfs. Les technologies d'apprentissage machine (ML) permettent de relever efficacement les défis en matière de soins de santé, tels que le suivi de l'état de santé des patients en évaluant la maladie d'un patient sur la base de ses symptômes et de ses dossiers. Les modèles d'apprentissage automatique analysent rapidement les données et génèrent des résultats. C'est pourquoi dans ce cadre des chercheurs comme Iqra Nissar dans l'article [1] ou Madhuri Kawarkhea dans l'article [2] ou Hualing Song dans l'article [3] ont élaboré dans leurs articles la solution d'utiliser l'apprentissage automatique comme moyen efficace pour obtenir des résultats d'analyse de manière automatique et rapide. Toutefois l'auteur Madhuri Kawarkhe dans l'article [2] mentionne que la production de résultats précis à l'aide de modèles de prédiction du diabète est une tâche difficile en raison de la disponibilité limitée des données et de l'existence de valeurs aberrantes. Raison pour laquelle lors de la construction du modèle d'apprentissage la phase de prétraitement de données et le choix de modèle doit être bien vérifié. Dans ce contexte nous nous sommes basés sur des données collectées auprès de la société irakienne, notamment au laboratoire du Medical City Hospital et du Centre spécialisé en endocrinologie et en diabétologie de l'hôpital universitaire d'Al-Kind afin de développer une application intuitive pouvant prédire la classe diabète d'un nouveau patient pour qu'il puisse obtenir le résultat de son état diabétique de manière rapide, fiable et efficace.

A. Objectif

L'objectif de ce projet est de développer une approche à long terme axées sur la technologie permettant à un décideur (médecin) de prédire la classe diabète d'un nouveau patient.

B. Solution proposée

Aujourd'hui, avec le développement rapide de la technologie, il existe des techniques permettant d'agir plus tôt avec un patient sous traitement de

diabète. Il s'agit par exemple de l'apprentissage automatique encore appelé Machine Learning qui a pour objectif principal d'apprendre le comportement d'une situation à partir d'étude faite sur des données à grande échelle afin de prédire le comportement d'autres nouvelles données de mêmes propriétés. Pour réaliser cela nous allons entraîner nos données avec des algorithmes d'apprentissage supervisé exigeant des points d'entrée ou caractéristique d'entrée ou X(feature) et un point de sortie ou y(target) tout en créant un modèle qui agit comme un être humain en prenant le relais du médecin pour exécuter de manière intelligent les tâches manuel que le médecin faisait. Il existe différents types d'algorithmes d'apprentissage supervisé. Le meilleur est celui qui donnera les meilleurs critères de performance sur nos données tels que : la précision, le recall, le record et le F1 Score etc...

C. Plan du projet

Ainsi pour la réalisation de ce travail, nous suivons les étapes suivantes:

- Présentation et préparation des données;
- L'importation des packages nécessaires pour coder avec le langage python;
- L'importation et affichage du jeu des données ;
- L'analyse exploratoire de ses données afin de mieux les comprendre et d'avoir des idées sur le traitement qui sera utile et nécessaire;
- Le prétraitement des données ;
- La modélisation des données ou nous avons utilisé plusieurs algorithmes de machine learning tels que : la régression logistique, la forêt aléatoire, le classificateur ou perceptron multicouche, et le support vector machine;

Et nous avons fini par une conclusion après avoir choisi le meilleur algorithme selon ses critères de performance.

II. RÉALISATION D'UN SYSTÈME INTELLIGENT POUR LE DIAGNOSTIC AUTOMATIQUE D'UN NOUVEAU DIABÈTE

A. Présentation et préparation des données

Les données qui ont été initialement saisies dans le système sont les suivantes :

- Numéro patients: représente le numéro unique de chaque patient .
 - Genre:Correspond au sexe des patients avec deux types de genre M ou F.
 - Âge: Correspond aux âges des patients représentés par des entiers naturels .
 - Urée (Urea): Correspond aux déchets azotés des patients représentés par des entiers relatifs.
 - Taux de créatinine (Cr): Le taux de créatinine sert à évaluer le fonctionnement des reins des patients représentés par des entiers naturels.
 - L'hémoglobine glyquée (HbA1c) : indique la moyenne de la glycémie des 2 à 3 derniers mois. Pour la majorité des adultes vivant avec le diabète, on vise une valeur d'A1C inférieure à 7,0 % et est représenté par des entiers relatifs.
 - Cholestérol (Chol): Représente le taux de cholestérol du patients et est représenté par des entiers relatifs.
 - Taux de sucre dans le sang (TG): représente le taux de glycémie et est représenté par des entiers relatifs.
 - HDL: lipoprotéines de haute densité, on l'appelle souvent bon cholestérol étant donné qu'un taux plus élevé de HDL peut réduire le risque de maladies et est représenté par des entiers relatifs.
 - LDL : C'est un type de cholestérol avec lequel son augmentation provoque le risque de maladie cardiovasculaire . Il est représenté par des entiers relatifs.
 - VLDL: Anomalie d'où son augmentation augmente le risque de diabète . Il est représenté par des entiers naturels .
 - BMI : Qui représente l'indice de masse corporelle du patient . Il est représenté par des entiers relatifs.
 - CLASS: Représente la variable cible et la classe de diabète du patient . Il est représenté par des caractères comme N (non diabétique) ou Y (diabétique) ou P prédit positifs .
- Conclusion : Comme la variable cible est catégorielle nous faisons face à un problème de classification .

B. Importation des outils nécessaires pour coder avec le langage python

[illegible]

C. Importation et affichage du jeux des données

The screenshot shows the JupyterLab interface. At the top, there's a menu bar with options like 'Fichier', 'Modifieur', 'Affichage', 'Insérer', 'Exécution', 'Outils', 'Aide', and a link to 'Dernière modification effectuée le 5 décembre'. Below the menu is a toolbar with icons for 'Code + Teste', 'Connecter', and 'Générer'. The main area is a code editor with the following code:

```
[ ] df = pd.read_csv('content/Dataset of Diabetes.csv')
```

Below the code editor, a table preview is shown with columns: ID, No_Patient, Gender, AGE, Urea, Cr, HbA1c, Cholest, TG, HDL, LDL, VLDL, BMT, CLASS. The first few rows of data are visible, showing patient information and their corresponding values for various health metrics.

D. Analyse exploratoire du jeu de données

1) *Information sur l'ensemble des données:*

The screenshot shows the Tableau interface with a table of data. The table has columns: ID, Nat_Person, AGE, Urea, Cr, HbA1c, Chol, TG, HDL, LDL, VLDL, and BMT. The data is sorted by ID in descending order. The interface includes a top bar with 'EXAMEN A UOOLYPS' and a sidebar with navigation icons.

ID	Nat_Person	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLDL	BMT
340.000000	1.000000+003	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
340.500000	2.705914+005	53.528000	5.124743	68.943000	8.281160	4.862820	2.348610	1.204750	2.609790	1.854700	29.978000
240.388773	3.350708+006	8.789241	2.935165	59.984747	2.534003	1.301738	1.401176	0.866144	1.115102	3.963599	4.962388
1.000000	1.230003+004	20.000000	0.500000	0.600000	0.000000	0.200000	0.300000	0.300000	0.100000	19.000000	19.000000
125.750000	2.406375+004	55.000000	3.700000	48.000000	6.500000	4.000000	1.500000	0.900000	1.800000	0.700000	26.000000
300.000000	3.349650+004	55.000000	4.600000	60.000000	4.800000	2.000000	1.100000	2.500000	0.900000	30.000000	30.000000
550.250000	4.538425+004	59.000000	5.700000	73.000000	10.200000	5.600000	2.900000	1.300000	3.300000	1.500000	33.000000
800.000000	7.543568+007	79.000000	38.900000	80.000000	16.000000	10.300000	13.800000	9.900000	8.900000	35.000000	47.750000

Remarque:

Nous remarquons que nos variables sont a différentes échelle tenant compte de leur max et de leur min, cela peut impacter négativement la performance des modèles de classification que nous allons construire . Raison pour laquelle dans la partie prétraitement nous allons standardiser ou normalisé nos variables.

2) *Présentation des variables d'entrées:*

The screenshot shows a JupyterLab environment. At the top, the file name is 'EXAMEN IA LQIQ.ipynb'. The top right bar contains icons for 'Rabbit Debugger' and 'Gerrit'. The left sidebar shows a file explorer with a folder named 'EXAMEN IA LQIQ'. The main area displays a Jupyter Notebook with a single cell containing the following code:

```
df.info()
```

The output of the code is a text representation of the DataFrame's information:

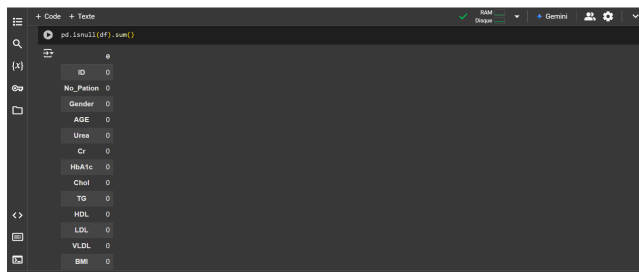
```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 1200 entries, 0 to 999
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype  
---  --   --
 0   ID             1200 non-null   int64   
 1   Occupation     1200 non-null   object  
 2   Gender         1200 non-null   int64   
 3   Age            1200 non-null   int64   
 4   Unwe           1200 non-null   float64  
 5   Cr             1200 non-null   int64   
 6   HbA1c          1200 non-null   float64  
 7   Chol           1200 non-null   float64  
 8   TG             1200 non-null   float64  
 9   HDL            1200 non-null   float64  
10   LDL            1200 non-null   float64  
11   VLDL           1200 non-null   float64  
12   BWE            1200 non-null   float64  
13   CLASES         1200 non-null   object  
dtypes: float64(8), int64(4), object(2)

```

Remarque:

Au totale nous avons quatorze variables dont deux variables catégorielles **GENDER** et **CLASS** et douze variables quantitatifs.



Remarque:

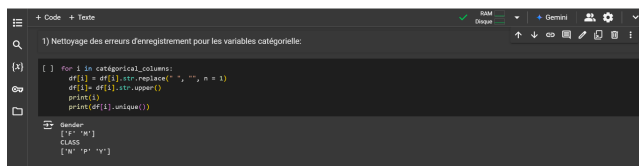
Apparemment il n'y a pas de valeurs manquantes.



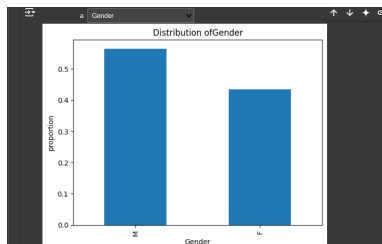
Remarque:

Nous remarquons des redondances de valeurs dans nos variables catégorielles. Cela peut impacter négativement la performance des modèles de classification que nous allons construire. Raison pour laquelle dans la suite nous allons Nettoyer ses erreurs.

Résultat après nettoyage:

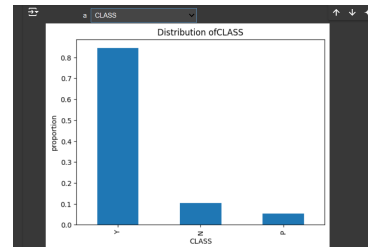


Affichage après nettoyage:



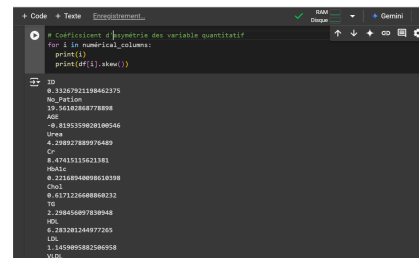
Remarque:

Nous remarquons avoir plus de males que de femelles.



Remarque:

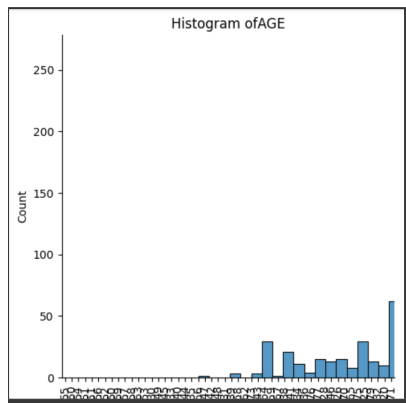
Pour la variable cible nous avons remarqué un problème de déséquilibre de la catégorie des patients de classe diabétique plus de 80% par rapport aux classes non diabétique moins de 12% et diabétique prédit moins de 6%. Cela peut impacter négativement la performance des modèles de classification que nous allons construire. Raison pour laquelle dans la suite nous allons procéder au sur-apprentissage ou au sur-apprentissage pour remédier à ce problème.



Remarque:

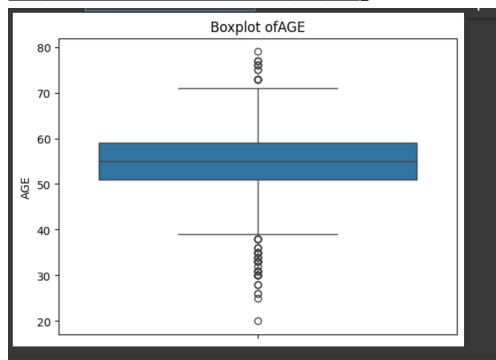
Nous remarquons que certaines de nos variables quantitatif ont un coefficient d'asymétrie <0 leurs distributions seront étalée à gauche de la moyenne et d'autres ont un coefficient d'asymétrie >0 leurs distributions seront étalée à droite de la moyenne.

Prenons l'exemple de la variable: AGE avec un coefficient d'asymétrie <0



La variable age a un coefficient <0 raison pour laquelle sa distribution est étalée à gauche de la moyenne. Cela peut impacter négativement la performance des modèles de classification que nous allons construire. Raison pour laquelle dans la suite nous allons remédier ses problèmes.

Nous allons utiliser des méthodes de transformation pour la transformer et la rendre beaucoup plus symétrique de même que toutes les autres variables asymétriques. Nous allons remédier à ce problème dans la phase de prétraitement.



Remarque: Nous remarquons des valeurs aberrantes aux extrémités des boîtes de certaines variables quantitatives. Et cela peut impacter négativement la performance des modèles de classification que nous allons construire. Prenons l'exemple de la variable age ci-dessus. Nous devons aussi appliquer des fonctions de transformation à ces variables afin de les rendre moins asymétriques afin de remédier à ce problème dans la phase de prétraitement.

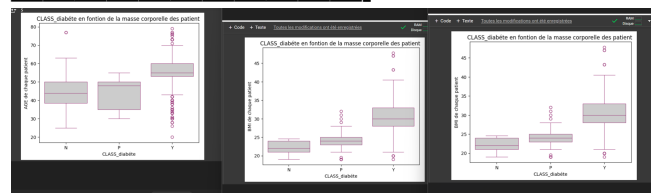
Conclusion:

Nous venons de faire une analyse univariée pour

comprendre chaque variable dans la suite nous allons procéder à l'analyse bi-variée pour identifier les corrélations qui existent entre les variables d'entrée et la variable de sortie. Comme un bon modèle est un modèle conçu avec des variables d'entrée en forte corrélation avec la variable de sortie.

3) Analyse bivariée sur les variables : —

Analyse de l'âge, du HbA1c, du BMI en fonction de la classe diabète



Remarque:

— Analyse de l'âge, du HbA1c, du BMI en fonction de la classe diabète prédictif

Nous remarquons que les patients testés positifs ont la plupart plus de 55 ans et sont plus minoritaires parmi les autres cas d'âges de patient. Un nombre important de patients âgés en moyenne de 49 ans risquent d'avoir le diabète. Un nombre important de patients âgés en moyenne de 42 ans mais moins importants que les patients âgés en moyenne de 50 ans sont testés négatifs.

Ce qui montre que la variable âge joue un rôle important pour la variable à prédire.

Analyse de l'HbA1c en fonction de la classe diabète

Nous remarquons que plus le HbA1c est important plus le patient est proche d'être diabétique positif. Un nombre important de patients testés positifs ont en moyenne un HbA1c égale à 9. Peu de patients ayant un HbA1c en moyenne égale à 7 risquent d'être positifs. Un nombre important de patients mais moins importants que ceux qui ont en moyenne un HbA1c égale à 9 et qui ont un HbA1c en moyenne égale à 5 sont testés négatifs.

Ce qui montre que la variable HbA1c joue un rôle important pour la variable à prédire.

Analyse du BMI en fonction de la classe diabète

Nous remarquons que plus le BMI est important plus le patient est ou proche d'être diabétique positif . Un nombre important de patient tester positif ont en moyenne un BMI égale a 31. Peu de patient ayant un BMI en moyenne égale a 23 risque d'être positif. Un nombre important de patient mais moins important que ceux qui ont en moyenne un BMI égale à 31 et qui ont un BMI en moyenne égale a 23 sont testé négatif.

Ce qui montre que la variable BMI joue un rôle important pour la variable à prédire .

— Conclusion de notre analyse exploratoire de données

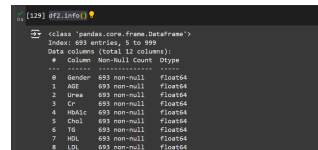
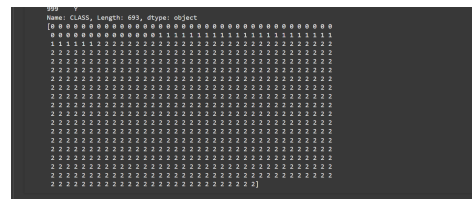
Cette analyse exploratoire de données nous a permis de mieux les comprendre nos données et d'avoir des indications sur le type de prétraitement que nous allons opéré avant de passer à la modélisation donc dans la suite du projet nous allons passer au prétraitement aux nettoyage de nos données.

E. Prétraitement des données

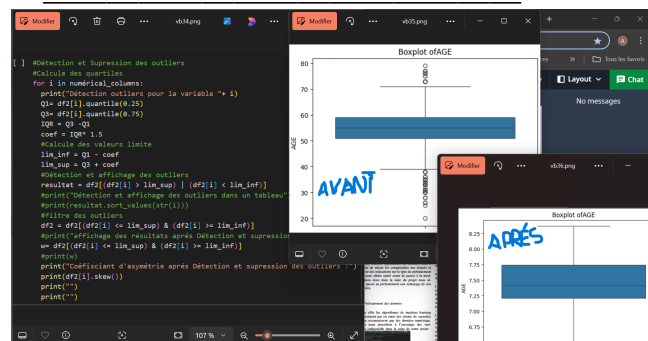
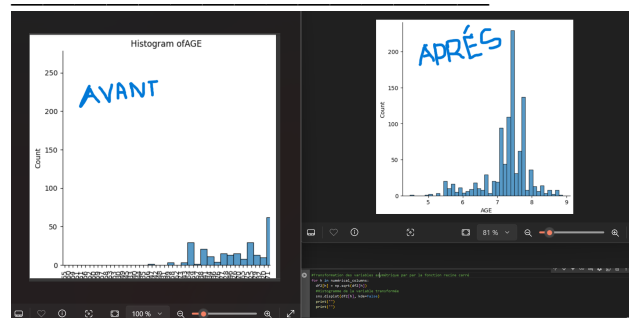
En effet les algorithmes de machine learning ne prennent pas en entrée des chaînes de caractère ils ne reconnaissent que des données numériques. D'où nous procédons à l'encodage des variables catégorielles dans la suite de notre projet . —

[illegible]

```
1100 x = df.drop('CLASS', axis=1)
1101 y = df['CLASS']
1102 from sklearn import preprocessing
1103 from sklearn import utils
1104 lab_enc = preprocessing.LabelEncoder()
1105 encoded = lab_enc.fit_transform(y)
1106 print('utils.MultiClassType of target')
1107 print('utils.MultiClassType of target + unique: %s' % y)
1108 print('utils.MultiClassType of target (encoded)')
1109 y = encoded
1110 print('df2["CLASS"]')
1111 print(y)
1112 seed = 1111
1113 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=4, stratify=y)
```



Essayons de rendre symétrie nos variables quantitatives et Supprimons les valeurs abérantes



Conclusion

Comme toutes nos variables sont maintenant numériques et symétrie maintenant nous pouvons procéder à la division du jeu de données .

F. Division en données d'entraînement de validation et de test du jeux de données obtenue après prétraitement

```
seed = 1111
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.4, random_state= seed, stratify= Y)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5, random_state= seed, stratify= y
```

Analyse

train-test-split est une fonction qui joue le rôle de division du jeu de données .

La variable **seed** est utilisée pour s'assurer de la reproductibilité des résultats parce que la fonction **train-test-split** fait la division des données de manière aléatoire pour s'assurer qu'à chaque nouvelle exécution qu'on garde la même division la variable **seed** est important.

La variable **stratify** permet de conserver la même proportion obtenue avant et après division des données de la variable cible. La fonction **train-test-split** fait cette division de manière aléatoire jusqu'à changer à chaque nouvelle exécution les proportions d'avant de nos variables donc les variables **seed** et **stratify** sont là pour remédier à ça .

J'ai 60% de données d'entraînement, 40% était attribué au jeu de test mais arrivé dans la partie de validation ce dernier est divisé en deux parties dont 20% seront maintenant attribués aux données de test et les 20% qui restent aux données de validation .

Cette validation sert à valider notre modèle sur des données qu'il n'a jamais vues afin de s'assurer sur ses performances à pouvoir faire de l'apprentissage sur des données futures .

G. Problème de déséquilibre de données

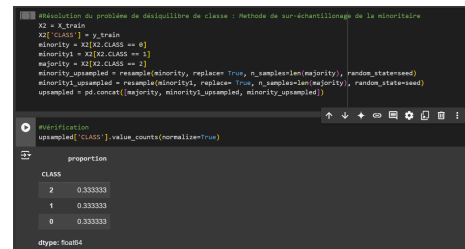
Lorsqu'il y a une grande différence entre le nombre d'observations dans chaque catégorie de la variable à prédire cela peut entraîner des erreurs de modélisation.

Dans notre cas ici, il y a 80% de patients testés positifs, à peu près 7% de patients prédits positifs et à peu près 4% de tests négatifs. Donc il y a un grand déséquilibre de classes . Nous pouvons utiliser le sur-échantillonnage pour créer plus d'équilibre entre les catégories de la variable cible. Soit on crée plus d'observations dans la classe minoritaire (modalité 1) c'est à dire on fait un sur-échantillonnage, soit on diminue les observations

de la classe majoritaire (modalité 0) c'est-à-dire un sous échantillonnage.

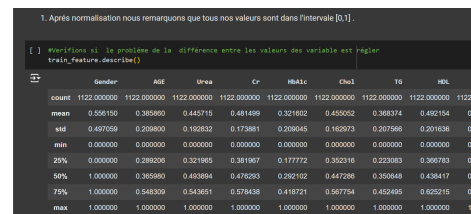
Remarque

Nous remarquons que le sur échantillonnage nous donne les meilleurs résultats que le sous échantillonnage . Raison pour laquelle nous avons décidé de continuer notre travail avec le sur échantillonnage.



H. Normalisation et Standardisation

Travaillons avec les données d'entraînement après un sur-échantillonnage puisque nous avons remarqué qu'il nous donne de meilleurs résultats contrairement au sous échantillonnage.



Conclusion

Donc le principe de la normalisation a réussi de mettre toutes les valeurs des variables au même niveau . Ainsi nous avons fini pour cette étape et dans les lignes qui suivent nous allons procéder à la partie modélisation.

I. Modélisation

Nous avons d'abord commencer par comprendre la problématique business, ensuite attaqué à l'analyse exploratoire de notre jeu de données pour mieux s'orienter, ensuite nous sommes passé à la préparation et nettoyage de nos données afin de pouvoir bien procéder à la partie modélisation.

L'accuracy: La precision globale du modèle est la proportion de prévision correctes, c'est à dire la somme du nombre de vrai positif(le patient est positif , la prediction prédit positif) et de vrai negatif(le patient est negatif , la prediction prédit negatif) divisé par le nombre totale d'observation. Un bon modèle est un modèle ou le diagonale TP et TF ont les plus grandes résultat.

Mais toute fois avoir un bonne accuray ne signifie pas avoir un bon modèle surtout quand il ya un problème de déséquilibre sur les données ciblé raison pour laquelle d'autre métrique comme recall et précision sont l'origine pour remédier a cette incohérence du bon modèle face un déséquilibre de données.

La précision: l'indicateur qui indique, sur tous les points positifs prédits , combien etaient de vrais positifs.

La Recall: métrique montrant la capacité du modèle à identifier tous les vrai positif .

Le problème avec ces deux métriques précision et recall est que si l'on fait pour améliorer l'un l'autre diminue .

Raison pour laquelle pour remédier a ce problème le métrique F1Score est née, ce métrique représente la moyenne entre une précision et le Recall .

Pour un modèle parfait, son f1-score doit est égale à 1 et la plus mauvaise performance est un modèle avec un f1-score égale à 0.

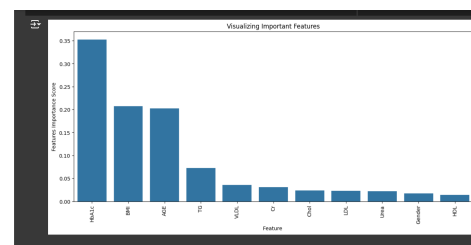
Nous choisissons le F1-score pour évaluer évaluer la performance de chaque modèle qui sera construit.

Conclusion:

Donc un meilleur algorithme est un algorithme avec un modèle qui renvoie le meilleur F1-Score et si on dit meilleur F1-score cela veut dire meilleur precision et meilleur recall puisqu'un F1-Score combine les deux . Dans les lignes qui suivent nous allons attaqué la partie modélisation afin d'entraîner nos données avec différentes type d'algorithmes et choisir celui qui nous donne un le modèle avec le meilleur F1-Score .

Avant de faire le choix d'algorithme nous allons selectionner les meilleurs variables prédictive:

Avec l'algorithme de RandomForest nous avons essayé d'afficher les meilleurs variable prédictive qui nous permet de faire une selection manuel et par la suite nous avons obtenue les résultats suivant:



Dans la suite de notre travail au lieu de selectionner manuellement les meilleurs variables prédictive afin d'optimiser le nbre de de caractéristique , nous avons choisit de faire une selection non manuel c'est à dire par RFE en utilisant une fonction de récursivité sélective.

Entraînement des algorithmes

Notre objectif est de construire un modèle de classification qui prédit la classe diabète d'un patient . Nous avons utiliser différents types d'algorithme à savoir (le Logistic Regression, Le Random For-

est, le Classificateur ou Perceptron Multicouche et le Support Vector Machine afin de comparer leurs performances et de choisir le meilleur modèle.

Modèle de regression logistique

```
. Modèle de regression logistique

# Définition des hyperparamètres
param_grid = {'C': [0.001, 0.01, 0.1, 1, 10, 100, 1000]}
# Création d'un objet GridSearchCV
grid_logreg_cv = GridSearchCV(estimator=LogisticRegression(random_state=seed), param_grid=param_grid, scoring='f1_macro')
# Entraînement de l'algorithme
logreg_model = grid_logreg_cv.fit(train_features, train_labels)
# Meilleur score et meilleur hyperparamètre
print(round(logreg_model.best_score_, 3))
print(logreg_model.best_estimator_)

0.984
LogisticRegression(C=1000, max_iter=1000, random_state=1111)

Le modèle a un bon score d'entraînement. Évaluons sa performance sur les données de validation afin d'apprécier sa capacité à généraliser sur de nouvelles données.

[] # Fonction d'évaluation de la performance d'un modèle
def model_evaluation(model, features, labels):
    prediction = model.predict(features)
    print(classification_report(labels, prediction))
```

Evaluation du modèle de regression logistique

```
# Évaluation du modèle de regression logistique
model_evaluation(logreg_model, X_val, y_val)

precision    recall  f1-score   support

0   0.77   1.00   0.87    10
1   0.33   0.67   0.44     6
2   0.99   0.92   0.95   124

accuracy: 0.78   0.80   0.76   140
macro avg: 0.73   0.83   0.83   140
weighted avg: 0.93   0.91   0.93   140
```

Logistic Régression avec RFE

```
# Création d'un objet de construction d'un modèle avec utilisation de l'algorithme RFE
def model_with_rfe(model):
    rfe_model = RFE(estimator=model, verbose=0)
    rfe_model.fit(train_features, train_labels)
    mask = rfe_model.support_
    reduced_X = train_features.iloc[:, mask]
    print(reduced_X.columns)
    return rfe_model

[] # Logistic regression avec RFE
rfe_logreg_model = model_with_rfe(logreg_model.best_estimator_)

Index(['bmi', 'bmi', 'tbs', 'choi', 'hdi'], dtype='object')

[] # Évaluation du modèle de regression logistique avec RFE
model_evaluation(rfe_logreg_model, X_val, y_val)

precision    recall  f1-score   support

0   0.75   0.90   0.82    10
1   0.33   0.67   0.44     6
2   0.99   0.92   0.95   124

accuracy: 0.78   0.80   0.76   140
macro avg: 0.73   0.83   0.83   140
```

Remarque

La RFE qui fait la sélection récursive automatique a réduit le nombre de variables prédictives de 11 à 5 et n'a pas amélioré la performance du modèle. Mais garde les mêmes performances avec le modèle sans sélection de variables.

Algorithme Random Forest et Evaluation du modèle de Random Forest

```
# Définition des hyperparamètres
param_grid_rf = {'n_estimators': [10, 50, 100, 500, 1000], 'max_depth': [3, 5, 10, 20, None]}
# Création d'un objet GridSearchCV
grid_rf_cv = GridSearchCV(estimator=RandomForestClassifier(random_state=seed), param_grid=param_grid_rf, scoring='f1_macro')
# Entraînement de l'algorithme
rf_model = grid_rf_cv.fit(train_features, train_labels)
# Meilleur score et meilleur hyperparamètre
print(round(rf_model.best_score_, 3))
print(rf_model.best_estimator_)

0.988
RandomForestClassifier(max_depth=10, n_estimators=500, random_state=1111)

[] # Évaluation du modèle de forêt aléatoire
model_evaluation(rf_model.best_estimator_, X_val, y_val)

precision    recall  f1-score   support

0   0.78   0.70   0.74    10
1   1.00   0.67   0.80     6
2   0.99   0.98   0.97   124

accuracy: 0.91   0.78   0.84   140
macro avg: 0.92   0.78   0.84   140
weighted avg: 0.97   0.87   0.92   140
```

Random Forest avec RFE

```
# Définition des hyperparamètres
rfe_rf_model = model_with_rfe(rf_model.best_estimator_)

Index(['bmi', 'bmi', 'tbs', 'choi', 'hdi'], dtype='object')

RFE
estimator: RandomForestClassifier

[] # Évaluation du modèle de forêt aléatoire avec RFE
model_evaluation(rfe_rf_model, X_val, y_val)

precision    recall  f1-score   support

0   0.80   0.70   0.75    10
1   1.00   0.67   0.80     6
2   0.99   0.98   0.98   124

accuracy: 0.92   0.79   0.85   140
macro avg: 0.93   0.78   0.85   140
weighted avg: 0.96   0.86   0.93   140
```

Remarque

La RFE qui fait la sélection récursive automatique a réduit le nombre de variables prédictives qui devient 5 l'utilisant avec RandomForest nous a permis d'avoir des résultats de F1-Score plus importants que les autres.

Support vecteur machine

```
# Définition des hyperparamètres
svm_model = SVC(random_state=seed)
svm_hyper = {'kernel': ['linear', 'rbf'], 'gamma': [0.01, 0.1, 1, 10, 100]}
# Création d'un objet GridSearchCV
svm_cv = GridSearchCV(svm_model, svm_hyper, cv=5, scoring='f1_macro')
# Entraînement de l'algorithme
svm_cv.fit(train_features, train_labels)
# Meilleur score et meilleur hyperparamètre
print(round(svm_cv.best_score_, 3))
print(svm_cv.best_estimator_)

1.0
SVC(gamma=100, random_state=1111)

[] # Évaluation du modèle de Support vecteur machine
model_evaluation(svm_cv.best_estimator_, X_val, y_val)

precision    recall  f1-score   support

0   0.00   0.00   0.00    10
1   1.00   0.17   0.20     6
2   0.99   1.00   0.99   124

accuracy: 0.63   0.39   0.41   140
macro avg: 0.63   0.39   0.41   140
weighted avg: 0.83   0.80   0.83   140
```

Conclusion

Nous avons utilisées les données d'évaluation pour sélectionner le meilleur modèle. Ensuite nous évaluons le meilleur modèle sélectionné sur les données de test afin d'apprécier sa performance sur de nouvelles données. Idéalement les performances de ce modèle sur les données d'évaluation et sur les données de test doivent être relativement proches.

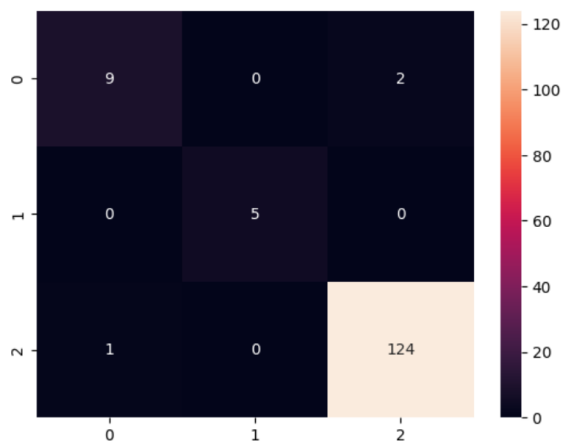
Nous avons remarqué que l'algorithme random-Forest avec RFE donnent les meilleurs citère de performance.

Evaluation de l'algorithme RandomForest avec les données test pour evaluer les performance du meilleur modèle

```
[ ]  RandomForest du meilleur modèle sur les données de test avec RandomForest
model_evaluation(rf_model.best_estimator_, X_test, Y_test)
```

	precision	recall	f1-score	support
0	0.00	0.00	0.00	11
1	1.00	1.00	1.00	5
2	0.00	0.00	0.00	125
accuracy	0.00	0.00	0.00	141
macro avg	0.00	0.00	0.00	141
weighted avg	0.00	0.00	0.00	141

Matrice de confusion du l'algorithme RandomForest



III. CONCLUSION

Nous avons utilisées les données d'évaluation pour sélectionner le meilleur modèle . Ensuite nous avons évaluer le meilleur modèle sélectionné sur les données de test afin d'apprécier sa performance sur de nouvelles données. Idéalement les performances de ce modèle sur les données d'évaluation et sur les données de test doivent être relativement proches.

Nous avons remarqué que l'algorithme random-Forest avec RFE donnent les meilleurs citère de performance.

REFERENCES

- [1] 1. Iqra Nissar¹ ,Jamia Millia Islamia, New Delhi, 110025, India; Waseem Ahmad Mir², GH Raison College of Engineering and Management, Pune, 412203, India; Tawseef Ayoub Shaikh ³, National Institute of Technology, Srinagar, 190006, India; Tuba Areen⁴, Maulana Mukhtar Ahmad Nadvi Technical Campus, Malegoan, 423203, India; Tuba Areen⁵, Sri Sai Group of Institutions, Punjab, India, 145001, India. Available online 31 May 2024, Version of Record 31 May 2024. <https://www.sciencedirect.com/science/article/pii/S1877050924009098?via=ih>
- [2] 2. Madhuri Kawarkhe¹ ; Research Scholar, MGM University, N6 Cidco, Aurangaba, 431003, MH India; Parminder Kaur² ;Professor, MGM University, N6 Cidco, Aurangaba, 431003, MH, India; Available online 31 May 2024, Version of Record 31 May 2024. <https://www.sciencedirect.com/science/article/pii/S1877050924007166?via=ih>
- [3] 3. Lijun Mao¹ ,School of Public Health, Shanghai University of Traditional Chinese Medicine, Shanghai, China; Luotao Lin ², Nutrition and Dietetics Program, Department of Individual, Family, and Community Education, University of New Mexico, United States; Zumin Shi ³, Human Nutrition Department, College of Health Sciences, QU Health, Qatar University, Qatar ; Xianglong Xu ⁴, School of Translational Medicine, Faculty of Medicine, Nursing and Health Sciences, Monash University, Melbourne, Victoria, Australia; Tuba Areen⁵, Sri Sai Group of Institutions, Punjab, India, 145001, India. Available online 31 May 2024, Version of Record 31 May 2024.